

Automated Root Cause Analysis Using a LangChain-Based RAG Anomaly Explanation Agent

Abstract

This article presents an AI-driven Root Cause Analysis (RCA) agent for MLOps monitoring environments. The proposed system automates the initial stage of incident investigation by responding to monitoring alerts, collecting supporting evidence from metrics, drift indicators, and dashboard observations, retrieving analogous historical incidents through a LangChain-based Retrieval-Augmented Generation (RAG) pipeline with Gemini embeddings and FAISS, and generating a structured RCA report with Gemini. The resulting report is stored as a Markdown file and delivered as a formatted HTML email, thereby reducing manual effort, improving reporting consistency, and enabling faster operational decision-making.

The workflow is initiated when Alertmanager sends an alert webhook to a FastAPI-based RCA service. The service orchestrates evidence collection, historical incident retrieval, and report generation in a fully automated sequence. Although the current prototype is demonstrated on a demand-type recall-drop scenario, the architecture is readily extensible to other monitoring alerts, incident categories, and MLOps use cases.

Index Terms—MLOps, Root Cause Analysis, Retrieval-Augmented Generation, LangChain, FAISS, Gemini, Prometheus, Alertmanager, FastAPI.

1. Introduction

Machine learning systems deployed in production require continuous monitoring to detect performance degradation, data quality issues, and service instability. Although monitoring platforms can identify abnormal conditions, they do not explain the underlying cause or indicate the most appropriate corrective action. In practice, Root Cause Analysis (RCA) remains largely manual and typically involves inspecting dashboards, reviewing logs, consulting historical incidents, and preparing summary reports.

This work addresses this gap by introducing an AI-driven RCA agent for MLOps environments. Upon receiving a monitoring alert, the system automatically collects evidence, retrieves relevant historical incidents using Retrieval-Augmented Generation (RAG), and produces a structured RCA report. The main contribution is the integration of monitoring, retrieval, and generative AI into a unified workflow that reduces investigation time and supports faster, more informed decision-making.

In this context, a **demand type** refers to an issue or service category associated with vehicle-related leads, such as battery-related, tire-related, or other maintenance-oriented cases, and is used for monitoring and downstream analysis. The RCA prototype is demonstrated on a demand-type recall-drop scenario because the underlying BMW MLOps system already organizes leads, monitoring outputs, and downstream reporting around these categories.

2. Proposed System Overview

The proposed system integrates monitoring, alerting, evidence collection, retrieval, and generative AI into a single RCA workflow. Prometheus continuously monitors model and service metrics, Alertmanager forwards alert webhooks, FastAPI coordinates the RCA stages, the retrieval layer identifies relevant historical incidents using Gemini embeddings and FAISS, and Gemini generates the structured RCA report. This design transforms a fragmented and manual investigation process into a structured and repeatable workflow that produces an actionable report for immediate operational use.

3. Proposed Methodology

Evidence Collection: The RCA service collects evidence relevant to the alert, including current metric values, feature drift indicators, dashboard observations, and alert metadata such as alert name, demand type, and severity. These inputs are consolidated into a structured evidence package for downstream analysis.

RAG Retrieval: Historical incidents are stored as text documents and transformed into vector embeddings using the Gemini embedding model. These embeddings are indexed in FAISS to support efficient similarity search over prior incident records. When a new incident occurs, the system retrieves the most contextually relevant cases and uses them as supporting context for downstream report generation.

Report Generation: The report generator combines the evidence package with the retrieved historical context to construct a structured prompt for the Gemini language model. The model then produces an RCA report covering the alert summary, probable root cause, supporting evidence, historical comparison, recommended actions, decision support, confidence level, and prevention guidance.

A key design choice in the methodology is that the generated RCA report is grounded in explicit evidence rather than produced from the alert text alone. The prompt construction stage therefore combines structured numerical evidence, retrieved historical narratives, and alert metadata in a controlled format. This grounding strategy improves interpretability, reduces unsupported explanations, and makes the generated report more useful in operational review settings.

4. Similarity Measure for Historical Incident Retrieval

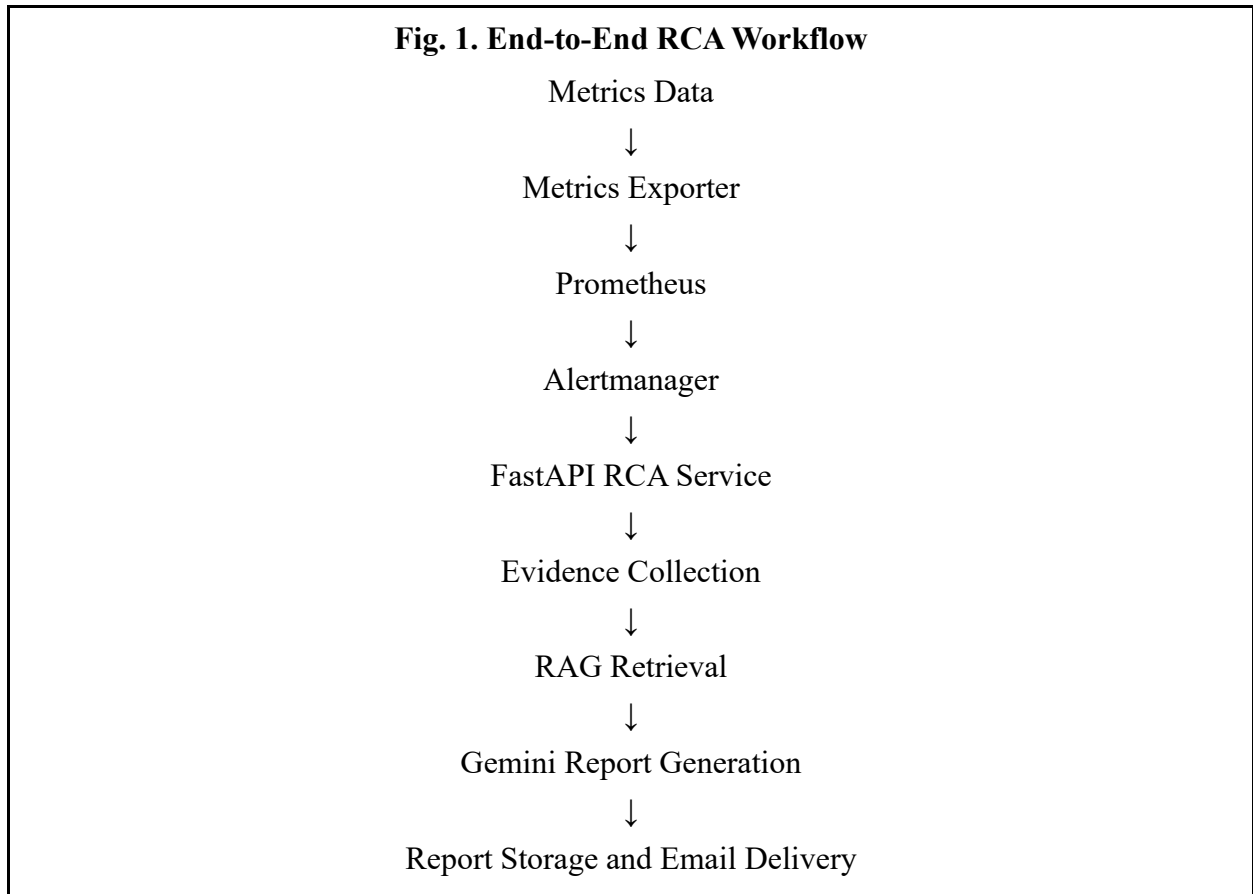
In the current prototype, historical incident retrieval is implemented using Gemini embeddings and a FAISS vector index through the LangChain-based retrieval pipeline. When a new incident query is constructed, it is embedded and searched against previously indexed incident embeddings to retrieve the top two most relevant incident chunks as contextual support for report generation. Conceptually, the similarity between a query embedding $\mathbf{q}$ and a document embedding $\mathbf{d}$ can be described using cosine similarity, defined in (1).

$$\text{cosine}(\mathbf{q}, \mathbf{d}) = (\mathbf{q} \cdot \mathbf{d}) / (\|\mathbf{q}\| \|\mathbf{d}\|) \quad (1)$$

In this expression, $\mathbf{q}$ represents the embedding of the current incident and $\mathbf{d}$ represents the embedding of a historical incident. A higher cosine similarity value indicates stronger contextual similarity between the two incidents. In the implemented system, the retriever returns the top two most similar results, and the retrieved incidents are inserted into the prompt sent to the report generator.

5. System Architecture and Workflow

The system architecture is organized as an alert-driven pipeline that links monitoring, evidence aggregation, retrieval, and AI-driven report generation into one continuous workflow. Prometheus continuously monitors exposed model and service metrics, Alertmanager routes significant alert events to the FastAPI RCA service, and the RCA service coordinates evidence collection, retrieval of analogous historical incidents, report generation, storage, and delivery. This modular architecture separates monitoring concerns from reasoning and reporting logic, which improves maintainability and makes the system easier to extend to additional alert types or models.



6. System Components

TABLE I
MAJOR COMPONENTS OF THE PROPOSED SYSTEM

Component	Technology	Purpose
Monitoring	Prometheus	Collects model and service performance metrics.
Alerting	Alertmanager	Triggers and routes alerts to the RCA service.
Dashboard	Grafana	Visualizes real-time system and model health.
API Layer	FastAPI	Orchestrates the end-to-end RCA pipeline.
Retrieval	LangChain and FAISS	Finds and ranks similar historical incidents.
AI Model	Google Gemini (LLM)	Generates embeddings and the structured RCA report.
Email Delivery	Gmail SMTP (App Password)	Sends the RCA report to the intended recipient.

These components operate as an interconnected stack rather than isolated services. Monitoring and alerting provide the trigger, the API layer coordinates execution, the retrieval layer injects historical memory into the process, and the language model converts both structured and unstructured evidence into a usable RCA narrative. Together, they form a pipeline that combines operational telemetry with AI-assisted reasoning.

7. Practical Example of Alert Processing

To illustrate the end-to-end behavior of the proposed system, consider a demand-type incident in which recall falls below the operational threshold. Prometheus detects the degradation through the exported metric stream, the alert is routed by Alertmanager, and the FastAPI RCA service is invoked automatically. The service then collects the latest evidence, including current metric values, drift signals, and dashboard observations, before retrieving the most relevant historical cases from the incident repository.

If the retrieved historical incident indicates a pattern of data-quality degradation rather than model obsolescence, the generated RCA report may recommend fixing the feature pipeline instead of retraining the model. This distinction can help guide engineering effort away from unnecessary retraining and toward the probable source of degradation. In this way, the system does not merely summarize an alert but also supports a more targeted and defensible corrective action.

TABLE II
ILLUSTRATIVE EXAMPLE OF ALERT PROCESSING

Stage	Illustrative Output
Alert Detection	Recall falls below the threshold for a monitored demand type.
Evidence Collection	Metrics, drift indicators, and dashboard observations are collected.
Historical Retrieval	A prior incident with similar data-quality symptoms is retrieved.
AI Report Generation	The RCA report may recommend fixing the feature pipeline instead of immediate retraining.
Operational Outcome	Stakeholders receive a structured RCA report with a recommended action.

This example demonstrates how the RCA system combines current evidence with historical incident context to recommend a likely corrective action, helping engineers focus on the probable source of degradation.

8. Conclusion

This article presented an AI-driven RCA agent for MLOps monitoring that automatically investigates model performance issues and provides structured explanations and recommendations. By combining monitoring alerts, evidence collection, retrieval-based historical reasoning, and generative AI, the system transforms RCA from a manual and fragmented activity into one that is rapid, consistent, and readily actionable. This work demonstrates how modern AI and retrieval techniques can be embedded into MLOps workflows to improve reliability, transparency, and response efficiency when managing machine learning systems at scale.